Assignment: Contoso Delay Classifier on Azure

Oct 13, 2026 · @Tiger

Build a working delay classifier on Azure, from SQL data to a live web portal, and submit it by Oct 16, 2026 (Friday, 11:59 PM IST).

Overview

You will take the Contoso order data, keep it in Azure SQL, train a model on Azure Machine Learning that predicts whether an order will be delayed, store the training results in Azure Cosmos DB, and put a web portal in front so anyone can submit an order and get a prediction. The portal also shows how healthy the system is: latency, error rate and a log of every request.

By the end you will have built, and be able to explain:

- a relational data layer (Azure SQL) and why it is the source of truth;
- an ML training run and an endpoint that serves it;
- a NoSQL store (Cosmos DB for NoSQL, queried with SQL syntax) for results and request logs, with a partition key you can defend;
- a front end and API that talk to the model without exposing any secret in the browser;
- basic operations skills: measuring latency and error rate, and tracking each user request.

Expect about 8 to 10 hours of work. Work alone or in pairs of two, as your instructor decides, and submit one repository per team.

How the pieces fit

Data flows one way through training and the other way through serving, and Cosmos DB sits in the middle collecting everything you want to look back at.

1. **Azure SQL** holds the order history (10,000 rows) in a table, with a view that returns the training columns.
2. A script reads the view and creates a training file, registered in **Azure Machine Learning** as a data asset.
3. An **AutoML classification job** (or your own scikit-learn job) trains models and picks the best one.
4. You deploy the best model to a **managed online endpoint** (REST URL plus key).

5. You write the **training results** (algorithm, AUC, accuracy, precision, recall, F1, endpoint name, date) into a Cosmos DB container called `model_runs`.
6. A **Static Web App** shows a form. The browser calls your own API (Azure Functions), never the model directly.
7. The API validates the input, calls the ML endpoint, **measures the time it took**, and writes one document per request into a Cosmos container called `requests`.
8. The API also exposes `/api/metrics`, which reads recent requests and returns **p50 and p95 latency, error rate and request count** for the portal's health panel.

The browser only knows your API address. The ML endpoint key and the Cosmos key stay in server-side settings, never in front-end code or in git.

Data provided

You get one file, `contoso_delay_training.csv`: 10,000 synthetic Contoso orders, of which 1,807 (18.1%) were delivered late. There is no personal data in it. Because only about one order in five is late, accuracy alone is a misleading score, so report AUC, precision and recall as well.

Column	Type	Meaning	Range or values
`carrier`	text	Courier used	Delhivery, BlueDart, EcomExp, IndiaPost
`distance_km`	integer	Warehouse to customer distance	45 to 6,073
`weight_kg`	decimal	Package weight	0.02 to 17.35
`items`	integer	Items in the order	1 to 7
`order_hour`	integer	Hour the order was placed	0 to 23
`is_sale_day`	0 or 1	Placed on a sale day	1 for about 11.5% of orders
`warehouse_backlog`	integer	Orders waiting at the warehouse	14 to 65
`weather_index`	decimal	Weather severity at destination	0.001 to 0.868
`delayed`	0 or 1	**Target**: delivered late	1 for 18.1% of orders

The file is attached by your instructor. If you cannot find it, ask in the class channel before you generate your own data.

Part A: Azure SQL (15 marks)

Load the CSV into Azure SQL, shape it for training, and prove you understand the data before you model it.

Tasks

1. Create an Azure SQL Database (serverless is fine) in your own resource group, with Entra-only authentication and your client IP allowed in the firewall.
2. Create the table below and load all 10,000 rows. Use any method: the Python hint below, SSMS or Azure Data Studio import, or `bcp`.
3. Create a schema `ml` and a view `ml.DelayTraining` that returns the nine columns and nothing else. Training code must read from the view, not the table.
4. Run the three analysis queries and keep a screenshot of each result.
5. Export `ml.DelayTraining` to a CSV or MLTable folder for Azure ML.

```
CREATE TABLE dbo.OrderHistory (
    OrderHistoryId    INT IDENTITY(1,1) PRIMARY KEY,
    carrier           NVARCHAR(20)  NOT NULL,
    distance_km       INT           NOT NULL,
    weight_kg         DECIMAL(6,2)  NOT NULL,
    items             INT           NOT NULL,
    order_hour        TINYINT       NOT NULL,
    is_sale_day       BIT           NOT NULL,
    warehouse_backlog INT           NOT NULL,
    weather_index     DECIMAL(5,3)  NOT NULL,
    delayed           BIT           NOT NULL
);
```

Analysis queries to write yourself

- Delay rate by carrier, highest first.
- Delay rate on sale days versus normal days.
- Delay rate by warehouse backlog bucket (for example under 30, 30 to 49, 50 and above).

In one paragraph, say which factor looks most related to delays and why you think so.

Hint for loading. In Python with `pyodbc` and `pandas`, connect with `Authentication=ActiveDirectoryInteractive`, turn on `cursor.fast_executemany = True`, and insert the rows in batches of 1,000. Do not commit the connection string or any

password to git.

Evidence to submit: the `CREATE` statements, row count (`SELECT COUNT(*)` returns 10,000), the three query results, and the paragraph.

Part B: Azure Machine Learning (20 marks)

Train a classifier that predicts `delayed`, judge it honestly, and deploy it as an API.

Tasks

1. Create an Azure ML workspace and a compute cluster with a minimum of 0 and a maximum of 1 node.
2. Register your export from Part A as a data asset. If you choose the Table (MLTable) type, upload a **folder** that holds both the CSV and a file named exactly `MLTable`. A lone CSV fails with a "StreamAccess.NotFound" error.
3. Train with **Automated ML**: task type Classification, target `delayed`, primary metric AUC weighted, a 30-minute limit. Advanced students may write their own scikit-learn training job instead.
4. Open the best model and record its metrics. You need at least AUC, accuracy, precision, recall and F1 for the delayed class, plus the confusion matrix.
5. Compare the best model with the second best in two sentences: is the difference worth caring about?
6. Deploy the best model to a **real-time endpoint** with key authentication, one instance, and a small VM size such as `Standard_DS2_v2`.
7. Test it from the Test tab with two orders you invent: one that should be on time and one that should be risky.
8. Copy the REST URL and key into your API's secure settings (Part D), never into the repo.

Questions to answer in your README

- The data is about 18% delayed. Why would a model that always says "on time" still score about 82% accuracy, and what does that tell you about choosing a metric?
- Which features did the model say matter most, and does that match your Part A analysis?
- If a customer is told "on time" and the order is late, what does that cost? Is the threshold right?

Note. A GUI-built AutoML endpoint normally returns the predicted class (0 or 1), not a probability. Your portal should show the class clearly. If you want a percentage, that is a bonus task (see Grading).

Evidence to submit: screenshot of the job leaderboard, the best model's metrics page, the endpoint Test tab with both sample orders, and your answers.

Part C: Cosmos DB for NoSQL (15 marks)

Store what the training run produced, and later what users do, in Cosmos DB, and query it with SQL syntax in Data Explorer.

Tasks

1. Create a Cosmos DB for NoSQL account in **serverless** mode, a database `contoso`, and two containers:

Container	Partition key	TTL	Holds
`model_runs`	`/modelName`	off	One document per training run or deployed model
`requests`	`/userId`	30 days	One document per portal request (Part D and E)

2. Write a short script (Python with the `azure-cosmos` package is fine) that reads the best model's metrics and **upserts** one document into `model_runs`. Use the schema below.
3. Run it twice with different data (for example the best and second-best model) so the container holds at least two runs.
4. In Data Explorer, run the three queries below and screenshot the results.
5. In four or five sentences, explain why you chose these partition keys and what would go wrong with `/carrier` or a constant key.

Document schema for `model_runs`

```
{
  "id": "run-2026-10-14-001",
  "modelName": "delay-classifier",
  "algorithm": "LightGBM",
  "datasetRows": 10000,
  "delayRate": 0.181,
  "metrics": { "auc": 0.0, "accuracy": 0.0, "precision": 0.0, "recall": 0.0,
"f1": 0.0 },
  "features": ["carrier", "distance_km", "weight_kg", "items", "order_hour",
"is_sale_day", "warehouse_backlog", "weather_index"],
  "endpointName": "delay-endpoint",
  "deployed": true,
  "trainedAt": "2026-10-14T10:30:00Z"
}
```

Replace the zeros with your real numbers. Do not invent values.

Queries to run in Data Explorer

```
SELECT c.id, c.algorithm, c.metrics.auc, c.trainedAt FROM c ORDER BY
c.metrics.auc DESC

SELECT VALUE COUNT(1) FROM c WHERE c.deployed = true

SELECT c.id, c.metrics.recall FROM c WHERE c.metrics.recall < 0.5
```

Also check the **RU charge** that Data Explorer shows for each query and write the three numbers down. Point reads (id plus partition key) cost less than queries; say in one line why.

Evidence to submit: the script, a screenshot of the `model_runs` documents, the query results with RU charges, and your partition-key explanation.

Part D: Web portal and API (20 marks)

Build a small website where a person fills in an order, clicks a button, and sees whether it is likely to be delayed. This is the part that makes the project real.

What to build

- **Front end:** a page hosted on Azure Static Web Apps, in plain HTML and JavaScript or a framework you already know.
- **API:** Azure Functions (Python or Node) deployed with the Static Web App, exposing `POST /api/predict`, `GET /api/metrics` and `GET /api/requests`.
- **Model:** the endpoint from Part B, called only from the API.
- **Store:** the `requests` container from Part C.

The page must have

1. A form with the eight input fields: a dropdown for carrier and number or toggle inputs for the rest, with sensible limits (for example order hour 0 to 23).
2. A **Predict** button that calls `/api/predict`.
3. A result card that says **"Likely delayed"** or **"Likely on time"**, plus the request ID and how long the call took in milliseconds.
4. A clear error message if anything fails (bad input, model down, timeout). The page must never show a raw stack trace.
5. A **health panel** and a **recent requests table** (Part E).
6. A fast first impression: loading spinner while waiting, disabled button while a request is running.

What `POST /api/predict` must do

1. Read or create a `userId` (the browser generates a random ID once and keeps it in `localStorage`, sent with every request).
2. Validate every field and reject bad input with HTTP 400 and a short reason. Do not call the model for invalid input.
3. Start a timer, call the ML endpoint with the key from server settings, and stop the timer. Use a timeout of about 10 seconds.
4. Map the model's answer to the label "delayed" or "on time".
5. Write one document to `requests` (schema in Part E) whether the call succeeded or failed.
6. Return JSON with `requestId`, `label`, `latencyMs` and `status`.

Security rules (graded)

- The ML key and Cosmos key live in the Static Web App's application settings or Key Vault. They are not in the browser code, not in the repo, and not in screenshots.
- Validate and sanitise input on the server even if the page already does it.
- Do not log personal data. The Contoso data is synthetic, and your logs should stay that way.
- Turn on HTTPS only (Static Web Apps does this for you).

Evidence to submit: the live URL, screenshots of a successful prediction and an error state, and the three API route files.

Part E: Latency, error rate and request tracking (15 marks)

A model nobody can measure is a demo, not a service. Add numbers to your portal so you can say how fast and how reliable it is, and so every request can be traced afterwards.

Document schema for `requests` (one per call, success or failure)

```
{
  "id": "b6f1c2a0-3c7e-4f0a-9d55-1a2b3c4d5e6f",
  "userId": "u-7f3a91",
  "at": "2026-10-15T09:12:41Z",
  "input": { "carrier": "IndiaPost", "distance_km": 2100, "weight_kg": 6.0,
"items": 3, "order_hour": 19, "is_sale_day": 1, "warehouse_backlog": 55,
"weather_index": 0.4 },
  "label": "delayed",
  "modelLatencyMs": 312,
  "totalLatencyMs": 348,
  "status": "ok",
  "httpStatus": 200,
  "error": null,
  "modelVersion": "delay-endpoint",
  "ttl": 2592000
}
```


`status` is `ok`, `invalid` (your 400 for bad input) or `error` (the model or a dependency failed). Keep `invalid` separate: bad user input is not a system failure.

Metrics the portal must show

Metric	How to compute it	Window
Requests	Count of documents	Last 15 minutes and all time
Latency p50	Median of `totalLatencyMs` for `ok` requests	Last 100 requests
Latency p95	95th percentile of the same list: sort ascending, take the value at position 0.95 × n	Last 100 requests
Slowest	Maximum `totalLatencyMs`	Last 100 requests
Error rate	`error` requests divided by (`ok` + `error`), as a percentage. Exclude `invalid`.	Last 15 minutes
Cold start	First request after 10 or more idle minutes, shown separately	Per session

Show a small line or bar chart of the last 50 latencies, and colour the error rate red when it goes above your own threshold (state the threshold you chose). Cosmos SQL has no percentile function, so compute p50 and p95 inside your API after reading the recent documents.

Tracking each user request

- Every response includes the `requestId`, and the page shows it.
- A **recent requests** table (last 20 for this `userId`) shows time, carrier, label, latency and status.
- A user can paste a request ID into a search box to look up one request. Use a point read (id plus `userId`) so it costs very little.

Two tests you must run and report

1. **Load test.** Send 100 requests with a short script (a Python loop with `requests` is enough; 5 at a time with a thread pool for extra credit). Report p50, p95, error rate and total time. State the VM size and instance count you used.
2. **Failure test.** Temporarily put a wrong key in the API settings, send 10 requests, and show that the error rate rises and the failures appear in `requests` with `status: error`. Restore the key and show the rate recovering. Screenshot both states.

Write a short paragraph on what you saw: where the time goes (network, model, database write), what the cold start cost, and what you would change first to make it faster.

Optional, high value: connect the Function app to **Application Insights** and show one chart of request duration and one of failures. A KQL query such as `requests | summarize avg(duration), count() by bin(timestamp, 5m)` is a good starting point.

Evidence to submit: screenshots of the health panel, the recent requests table, the load-test output, both failure-test states, and your paragraph.

Submission

Submit everything by **Friday 16 October 2026, 11:59 PM IST**. Late work loses 10 marks per day unless you have told the instructor before the deadline.

What to hand in

1. A **GitHub repository** (invite the instructor if it is private) with the layout below.
2. The **live portal URL**, working on the day of grading.
3. A **3 to 5 minute screen recording** that shows: a prediction in the portal, the same request in Cosmos Data Explorer, the health panel, and the failure test.
4. A **README** with your architecture diagram, setup steps, the answers to the questions in Parts A to E, and a short reflection (what was hardest, what you would do differently, what you would add for production).

Suggested repository layout

```
contoso-delay-assignment/
  README.md
  sql/            create_table.sql, view.sql, analysis.sql
  ml/             export_data.py, MLTable, notes.md (metrics, comparison)
  cosmos/         push_results.py, queries.sql
  app/            index.html, app.js, styles.css
  api/            predict/, metrics/, requests/
  tests/          load_test.py
  screenshots/    01-sql-count.png ...
  .gitignore      (must exclude .env, local.settings.json, keys)
```

Screenshot checklist

- [ ] SQL row count of 10,000 and the three analysis results
- [ ] Azure ML job leaderboard and best-model metrics
- [ ] Endpoint Test tab with a risky and a safe order
- [ ] Cosmos `model_runs` documents and the three query results with RU charge
- [ ] Portal: successful prediction, validation error, model-failure error
- [ ] Health panel with p50, p95, error rate and request count
- [ ] Recent requests table and one request found by ID
- [ ] Load-test output, and the failure test before and after

Rules

- **No secrets in git.** If a key is ever committed, rotate it at once and say so in the README. Check your history before you submit.
- **Cost.** Use one resource group, add a budget alert, and keep the ML endpoint running only while you test and record. Delete the endpoint (and the Redis or any other extras) when you finish, and delete the resource group after grading. The endpoint VM bills by the hour even when idle.
- **Region and quota.** If a VM size or model is not available in your region, pick the closest option and write what you changed.
- **Questions.** Ask in the class channel by Wednesday evening so everyone sees the answer.

Grading, bonus and integrity

The assignment is marked out of 100, with up to 15 bonus marks for going further.

Area	Marks	Full marks look like
Part A: Azure SQL	15	Correct table and view, 10,000 rows loaded, three analysis queries, a sensible conclusion
Part B: Azure ML	20	Honest metrics beyond accuracy, a working endpoint, thoughtful answers about the imbalance and threshold
Part C: Cosmos DB	15	Both containers, a defended partition key, real results stored, queries with RU charges
Part D: Portal and API	20	Working form and result, input validation, clean error handling, no secrets exposed
Part E: Observability	15	Correct p50, p95 and error rate, request tracking, load test and failure test with evidence
Documentation and cost control	10	Clear README, architecture diagram, budget alert, resources cleaned up
Demo video	5	Clear, short, shows the whole flow

Bonus ideas (up to 15 marks)

- Return a **probability** by training your own model and deploying it so that it returns class probabilities, and show a risk meter on the page (5).
- Add **Application Insights** with a dashboard of duration and failures (4).
- Add a **Redis cache** for repeated identical requests and show the latency difference (3).
- Add the **Cosmos change feed** so a delayed prediction creates a document in a `review_queue` container (3).

Integrity. You may use documentation, AI assistants and class notes, but you must understand everything you submit and be able to explain it. Expect a short oral question or two during grading, such as "why this partition key?" or "where does the time go in a request?". Teams may discuss ideas, but each team writes and runs its own code on its own subscription. Copied repositories will receive zero.